

PLSQL

Exercise 4: Functions

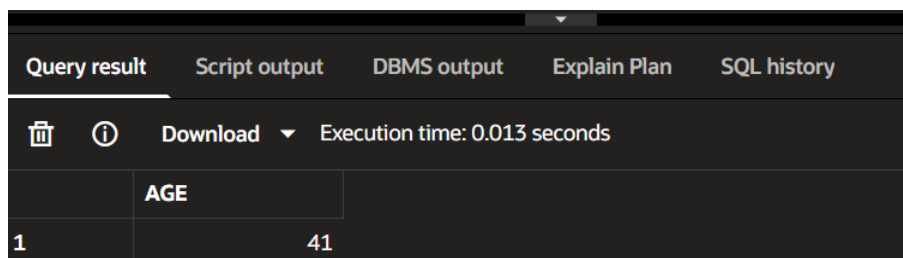
Scenario 1:

```
CREATE OR REPLACE FUNCTION CalculateAge(  
    p_dob DATE  
)  
  
    RETURN NUMBER  
  
IS  
  
    v_age NUMBER;  
  
BEGIN  
  
    v_age := FLOOR(  
        MONTHS_BETWEEN(  
            SYSDATE,  
            p_dob  
        ) / 12  
    );  
  
    RETURN v_age;  
  
END;  
  
/
```

Step 2: Test Function

```
SELECT CalculateAge(  
    TO_DATE('1985-05-15','YYYY-MM-DD')  
) AS Age  
FROM Dual;
```

Output



The screenshot shows a database query result window with a dark theme. At the top, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. Below the tabs, there is a toolbar with icons for a table, an information icon, a 'Download' button, and a dropdown arrow. To the right of the toolbar, it says 'Execution time: 0.013 seconds'. The main area displays a table with one column named 'AGE' and one row with the value '41'.

	AGE
1	41

Use with Customers Table

```
SELECT CustomerID,  
    Name,
```

```
CalculateAge(DOB) AS Age
FROM Customers;
```

Sample Output

Query result Script output DBMS output Explain Plan SQL history				
  Download ▾ Execution time: 0.007 seconds				
	CUSTOMERID	NAME	AGE	
1	1	John Doe	41	
2	2	Jane Smith	35	
3	5	David Miller	28	
4	3	Robert King	76	

Scenario 2:

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
  p_loan_amount NUMBER,
  p_interest_rate NUMBER,
  p_years NUMBER
)
RETURN NUMBER
IS
  v_monthly_installment NUMBER;
BEGIN

  v_monthly_installment :=
    (p_loan_amount +
     (p_loan_amount * p_interest_rate * p_years / 100))
    / (p_years * 12);

  RETURN ROUND(v_monthly_installment,2);

END;
/
```

Test Function

```
SELECT CalculateMonthlyInstallment(
  100000,
  10,
  5
) AS EMI
FROM Dual;
```

Query result		Script output	DBMS output
  Download ▼		Execution time: 0.007 se	
	EMI		
1	2500		

Scenario 3:

```
CREATE OR REPLACE FUNCTION HasSufficientBalance(
```

```
    p_account_id NUMBER,
```

```
    p_amount NUMBER
```

```
)
```

```
RETURN VARCHAR2
```

```
IS
```

```
    v_balance NUMBER;
```

```
BEGIN
```

```
    SELECT Balance
```

```
    INTO v_balance
```

```
    FROM Accounts
```

```
    WHERE AccountID = p_account_id;
```

```
    IF v_balance >= p_amount THEN
```

```
        RETURN 'TRUE';
```

```
    ELSE
```

```
        RETURN 'FALSE';
```

```
    END IF;
```

EXCEPTION

WHEN NO_DATA_FOUND THEN

RETURN 'ACCOUNT NOT FOUND';

END;

Test Function

```
SELECT HasSufficientBalance(  
    1,  
    500  
    ) AS Result  
FROM Dual;
```

Output

Query result		Script output	DBMS output	Explain Plan	SQL history
		Download ▾ Execution time: 0.005 seconds			
		RESULT			
1		TRUE			

Test Insufficient Balance

```
SELECT HasSufficientBalance(  
    1,  
    5000  
    ) AS Result  
FROM Dual;
```

Output

Query result		Script output	DBMS output	Explain Plan	SQL history
		Download ▾ Execution time: 0.01 seconds			
		RESULT			
1		FALSE			

Test Invalid Account

```
SELECT HasSufficientBalance(
    100,
    500
) AS Result
FROM Dual;
```

Output

Query result		Script output	DBMS output	Explain Plan	SQL history
 		Download ▾ Execution time: 0.005 seconds			
	RESULT				
1	ACCOUNT NOT FOUND				

Exercise 5: Triggers

Scenario 1: CREATE OR REPLACE TRIGGER UpdateCustomerLastModified

BEFORE UPDATE

ON Customers

FOR EACH ROW

BEGIN

 :NEW.LastModified := SYSDATE;

END;

/

Check Current Data

```
SELECT CustomerID,
    Name,
    LastModified
FROM Customers;
```

Update a Customer

```
UPDATE Customers
SET Balance = 5000
WHERE CustomerID = 1;
```

COMMIT;

Step 4: Verify

```

SELECT CustomerID,
       Name,
       Balance,
       LastModified
FROM Customers
WHERE CustomerID = 1;

```

Output:

Query result				Script output	DBMS output	Explain Plan	SQL history
				Download			
				Execution time: 0.006 seconds			
	CUSTOMERID	NAME	LASTMODIFIED				
1		1 John Doe	6/24/2026, 10:16:59 AM				
2		2 Jane Smith	6/24/2026, 10:16:59 AM				
3		5 David Miller	6/24/2026, 10:35:24 AM				
4		3 Robert King	6/24/2026, 10:19:36 AM				

Scenario 2:

Create AuditLog Table

```

CREATE TABLE AuditLog (
    LogID NUMBER GENERATED BY DEFAULT AS IDENTITY,
    TransactionID NUMBER,
    ActionDate DATE,
    ActionType VARCHAR2(50)
);

```

Create Trigger

```

CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT
ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog(
        TransactionID,
        ActionDate,
        ActionType
    )
    VALUES(
        :NEW.TransactionID,
        SYSDATE,
        'INSERT'
    );

```

```
END;  
/
```

Insert a Transaction

```
INSERT INTO Transactions  
VALUES(  
    10,  
    1,  
    SYSDATE,  
    500,  
    'Deposit'  
);
```

```
COMMIT;
```

Verify

```
SELECT *  
FROM AuditLog;
```

Output

Query result Script output DBMS output Explain Plan SQL history				
		Download ▼	Execution time: 0.012 seconds	
	LOGID	TRANSACTIONID	ACTIONDATE	ACTIONTYPE
1	1	10	6/24/2026, 11:04:35 AM	INSERT

Scenario 3:

Create Trigger

```
CREATE OR REPLACE TRIGGER CheckTransactionRules  
BEFORE INSERT  
ON Transactions  
FOR EACH ROW  
DECLARE  
    v_balance NUMBER;  
BEGIN  
  
    SELECT Balance  
    INTO v_balance  
    FROM Accounts
```

```

WHERE AccountID = :NEW.AccountID;

IF :NEW.TransactionType = 'Withdrawal' THEN

    IF :NEW.Amount > v_balance THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Withdrawal exceeds account balance'
        );
    END IF;

ELSIF :NEW.TransactionType = 'Deposit' THEN

    IF :NEW.Amount <= 0 THEN
        RAISE_APPLICATION_ERROR(
            -20002,
            'Deposit amount must be positive'
        );
    END IF;

END IF;

END;
/

```

Test Valid Deposit

```

INSERT INTO Transactions
VALUES(
    11,
    1,
    SYSDATE,
    100,
    'Deposit'
);
COMMIT;

```

Step 3: Test Invalid Deposit

```

INSERT INTO Transactions
VALUES(
    12,
    1,
    SYSDATE,
    -100,
    'Deposit'
);

```

Output

```

ORA-20002: Deposit amount must be positive
ORA-06512: at "MADHUMITAA_BTМ_SCHEMA_XV86T.CHECKTRANSACTIONRULES", line 22
ORA-04088: error during execution of trigger 'MADHUMITAA_BTМ_SCHEMA_XV86T.CHECKTRANSACTIONRULES'

```


Test Invalid Withdrawal

Suppose Account 1 balance is 1000.

```
INSERT INTO Transactions
VALUES(
  13,1,
  SYSDATE,
  5000,
  'Withdrawal'
);
```

Output

```
ORA-20001: Withdrawal exceeds account balance
ORA-06512: at "MADHUMITAA_BT_M_SCHEMA_XV86T.CHECKTRANSACTIONRULES", line 13
ORA-04088: error during execution of trigger 'MADHUMITAA_BT_M_SCHEMA_XV86T.CHECKTRANSACTIONRULES'
```

Exercise 6: Cursors

Scenario 1:

DECLARE

CURSOR GenerateMonthlyStatements IS

```
SELECT c.CustomerID,
       c.Name,
       t.TransactionID,
       t.TransactionDate,
       t.Amount,
       t.TransactionType
```

FROM Customers c

JOIN Accounts a

ON c.CustomerID = a.CustomerID

JOIN Transactions t

ON a.AccountID = t.AccountID

WHERE EXTRACT(MONTH FROM t.TransactionDate)

```

        = EXTRACT(MONTH FROM SYSDATE)

        AND EXTRACT(YEAR FROM t.TransactionDate)

        = EXTRACT(YEAR FROM SYSDATE);

v_rec GenerateMonthlyStatements%ROWTYPE;

BEGIN

    OPEN GenerateMonthlyStatements;

    LOOP

        FETCH GenerateMonthlyStatements

        INTO v_rec;

        EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(

            'Customer: ' || v_rec.Name ||

            ' | Transaction ID: ' || v_rec.TransactionID ||

            ' | Amount: ' || v_rec.Amount ||

            ' | Type: ' || v_rec.TransactionType

        );

    END LOOP;

    CLOSE GenerateMonthlyStatements;

END;

/

```

Output:



```
Customer: John Doe | Transaction ID: 11 | Amount: 100 | Type: Deposit
Customer: John Doe | Transaction ID: 1 | Amount: 200 | Type: Deposit
Customer: Jane Smith | Transaction ID: 2 | Amount: 300 | Type: Withdrawal
Customer: John Doe | Transaction ID: 10 | Amount: 500 | Type: Deposit
```

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.027
```

Scenario 2:

DECLARE

CURSOR ApplyAnnualFee IS

SELECT AccountID,

Balance

FROM Accounts;

v_accountid Accounts.AccountID%TYPE;

v_balance Accounts.Balance%TYPE;

BEGIN

OPEN ApplyAnnualFee;

LOOP

FETCH ApplyAnnualFee

INTO v_accountid,

v_balance;

EXIT WHEN ApplyAnnualFee%NOTFOUND;

UPDATE Accounts

SET Balance = Balance - 100

WHERE AccountID = v_accountid;

DBMS_OUTPUT.PUT_LINE(

```

        'Annual Fee Applied to Account '

        || v_accountid

    );

END LOOP;

CLOSE ApplyAnnualFee;

COMMIT;

END;

/
SELECT AccountID,

        Balance

FROM Accounts;

```

Output:

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.001 seconds

	ACCOUNTID	BALANCE
1	1	400
2	2	3900

Scenario 3: UpdateLoanInterestRates

```

DECLARE

    CURSOR UpdateLoanInterestRates IS

        SELECT LoanID,

                InterestRate

        FROM Loans;

    v_loanid Loans.LoanID%TYPE;

```

```

v_rate Loans.InterestRate%TYPE;

BEGIN

OPEN UpdateLoanInterestRates;

LOOP

    FETCH UpdateLoanInterestRates

    INTO v_loanid,

        v_rate;

    EXIT WHEN UpdateLoanInterestRates%NOTFOUND;

    UPDATE Loans

    SET InterestRate = InterestRate - 0.5

    WHERE LoanID = v_loanid;

    DBMS_OUTPUT.PUT_LINE(

        'Interest Rate Updated for Loan '

        || v_loanid

    );

END LOOP;

CLOSE UpdateLoanInterestRates;

COMMIT;

END;
SELECT LoanID,

    InterestRate

FROM Loans; /

```

Output:

Query result

Script output

DBMS output

Explain Plan

SQL history





Download

▼

Execution time: 0.006 seconds

	LOANID	INTERESTRATE
1	1	4.5
2	2	-2.5

Exercise 7: Packages

Scenario 1

CREATE OR REPLACE PACKAGE CustomerManagement AS

```

PROCEDURE AddCustomer(
    p_CustomerID NUMBER,
    p_Name VARCHAR2,
    p_DOB DATE,
    p_Balance NUMBER
);

```

```

PROCEDURE UpdateCustomer(
    p_CustomerID NUMBER,
    p_Name VARCHAR2,
    p_Balance NUMBER
);

```

```

FUNCTION GetCustomerBalance(
    p_CustomerID NUMBER
) RETURN NUMBER;

```

END CustomerManagement;

/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

```

PROCEDURE AddCustomer(
    p_CustomerID NUMBER,
    p_Name VARCHAR2,
    p_DOB DATE,
    p_Balance NUMBER
) IS
BEGIN
    INSERT INTO Customers
    VALUES(
        p_CustomerID,
        p_Name,
        p_DOB,

```

```

        p_Balance,
        SYSDATE
    );

    DBMS_OUTPUT.PUT_LINE('Customer Added Successfully');
END AddCustomer;

PROCEDURE UpdateCustomer(
    p_CustomerID NUMBER,
    p_Name VARCHAR2,
    p_Balance NUMBER
) IS
BEGIN
    UPDATE Customers
    SET Name = p_Name,
        Balance = p_Balance,
        LastModified = SYSDATE
    WHERE CustomerID = p_CustomerID;

    DBMS_OUTPUT.PUT_LINE('Customer Updated Successfully');
END UpdateCustomer;

FUNCTION GetCustomerBalance(
    p_CustomerID NUMBER
) RETURN NUMBER
IS
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Customers
    WHERE CustomerID = p_CustomerID;

    RETURN v_Balance;
END GetCustomerBalance;

END CustomerManagement;
/

```

Execution

```

BEGIN
    CustomerManagement.AddCustomer(
        4,
        'David Miller',
        TO_DATE('1995-08-10','YYYY-MM-DD'),
        2500
    );
END;
/

```

Output:

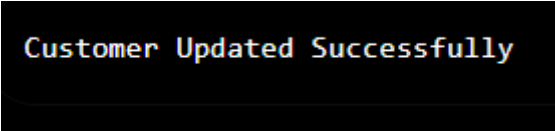
Customer Added Successfully

```

BEGIN
  CustomerManagement.UpdateCustomer(
    4,
    'David M',
    3000
  );
END;
/

```

Output:



```

Customer Updated Successfully

```

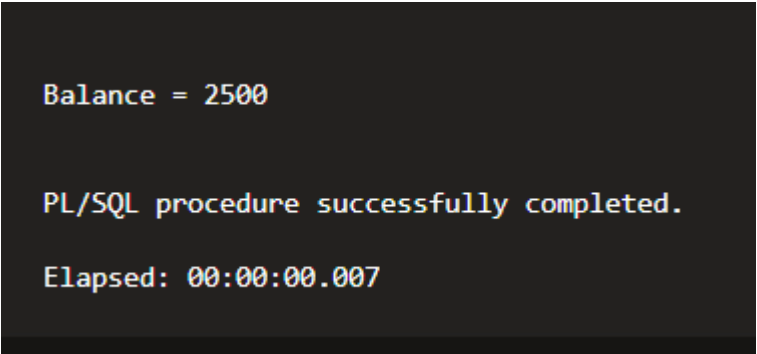
```

DECLARE
  v_balance NUMBER;
BEGIN
  v_balance := CustomerManagement.GetCustomerBalance(4);

  DBMS_OUTPUT.PUT_LINE(
    'Balance = ' || v_balance
  );
END;
/

```

Output:



```

Balance = 2500

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.007

```

Scenario 2:

CREATE OR REPLACE PACKAGE EmployeeManagement AS

```

PROCEDURE HireEmployee(
  p_EmployeeID NUMBER,
  p_Name VARCHAR2,
  p_Position VARCHAR2,
  p_Salary NUMBER,
  p_Department VARCHAR2,
  p_HireDate DATE
);

```

```

PROCEDURE UpdateEmployee(

```



```

        p_EmployeeID NUMBER,
        p_Position VARCHAR2,
        p_Salary NUMBER
    );

    FUNCTION CalculateAnnualSalary(
        p_EmployeeID NUMBER
    ) RETURN NUMBER;

END EmployeeManagement;
/
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee(
        p_EmployeeID NUMBER,
        p_Name VARCHAR2,
        p_Position VARCHAR2,
        p_Salary NUMBER,
        p_Department VARCHAR2,
        p_HireDate DATE
    ) IS
    BEGIN
        INSERT INTO Employees
        VALUES(
            p_EmployeeID,
            p_Name,
            p_Position,
            p_Salary,
            p_Department,
            p_HireDate
        );

        DBMS_OUTPUT.PUT_LINE('Employee Hired Successfully');
    END HireEmployee;

    PROCEDURE UpdateEmployee(
        p_EmployeeID NUMBER,
        p_Position VARCHAR2,
        p_Salary NUMBER
    ) IS
    BEGIN
        UPDATE Employees
        SET Position = p_Position,
            Salary = p_Salary
        WHERE EmployeeID = p_EmployeeID;

        DBMS_OUTPUT.PUT_LINE('Employee Updated Successfully');
    END UpdateEmployee;

    FUNCTION CalculateAnnualSalary(
        p_EmployeeID NUMBER
    ) RETURN NUMBER
    IS
        v_salary NUMBER;
    BEGIN
        SELECT Salary
        INTO v_salary

```

```

        FROM Employees
        WHERE EmployeeID = p_EmployeeID;

        RETURN v_salary * 12;
    END CalculateAnnualSalary;

END EmployeeManagement;
/

```

Execution & Output

```

BEGIN
    EmployeeManagement.HireEmployee(
        3,
        'Charlie',
        'Analyst',
        50000,
        'Finance',
        SYSDATE
    );
END;
/

```

Output:

```
Employee Hired Successfully
```

```

BEGIN
    EmployeeManagement.UpdateEmployee(
        3,
        'Senior Analyst',
        55000
    );
END;
/

```

Output:

```
Employee Updated Successfully
```

```

DECLARE
    v_annual NUMBER;
BEGIN
    v_annual :=
        EmployeeManagement.CalculateAnnualSalary(3);

    DBMS_OUTPUT.PUT_LINE(
        'Annual Salary = ' || v_annual
    );
END;
/

```

Annual Salary = 660000

Scenario 3:

CREATE OR REPLACE PACKAGE AccountOperations AS

```
PROCEDURE OpenAccount(  
    p_AccountID NUMBER,  
    p_CustomerID NUMBER,  
    p_AccountType VARCHAR2,  
    p_Balance NUMBER  
);
```

```
PROCEDURE CloseAccount(  
    p_AccountID NUMBER  
);
```

```
FUNCTION GetTotalBalance(  
    p_CustomerID NUMBER  
) RETURN NUMBER;
```

END AccountOperations;

/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

```
PROCEDURE OpenAccount(  
    p_AccountID NUMBER,  
    p_CustomerID NUMBER,  
    p_AccountType VARCHAR2,  
    p_Balance NUMBER  
) IS
```

BEGIN

INSERT INTO Accounts

VALUES(

p_AccountID,

p_CustomerID,

p_AccountType,

p_Balance,

SYSDATE

);

DBMS_OUTPUT.PUT_LINE('Account Opened Successfully');

END OpenAccount;

```
PROCEDURE CloseAccount(  
    p_AccountID NUMBER  
) IS
```

BEGIN

DELETE FROM Accounts

WHERE AccountID = p_AccountID;

```

        DBMS_OUTPUT.PUT_LINE('Account Closed Successfully');
    END CloseAccount;

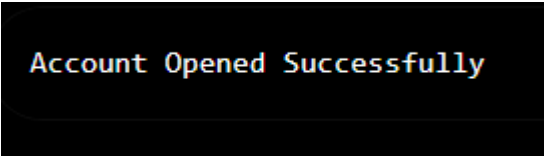
    FUNCTION GetTotalBalance(
        p_CustomerID NUMBER
    ) RETURN NUMBER
    IS
        v_Total NUMBER;
    BEGIN
        SELECT NVL(SUM(Balance),0)
        INTO v_Total
        FROM Accounts
        WHERE CustomerID = p_CustomerID;

        RETURN v_Total;
    END GetTotalBalance;

END AccountOperations;
/
BEGIN
    AccountOperations.OpenAccount(
        3,
        1,
        'Savings',
        5000
    );
END;
/

```

Output:



```

Account Opened Successfully

```

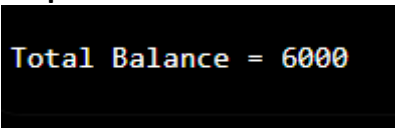
```

DECLARE
    v_total NUMBER;
BEGIN
    v_total := AccountOperations.GetTotalBalance(1);

    DBMS_OUTPUT.PUT_LINE(
        'Total Balance = ' || v_total
    );
END;
/

```

Output:



```

Total Balance = 6000

```

```

BEGIN
    AccountOperations.CloseAccount(3);

```

```
END;  
/
```

Output:

```
Account Closed Successfully
```